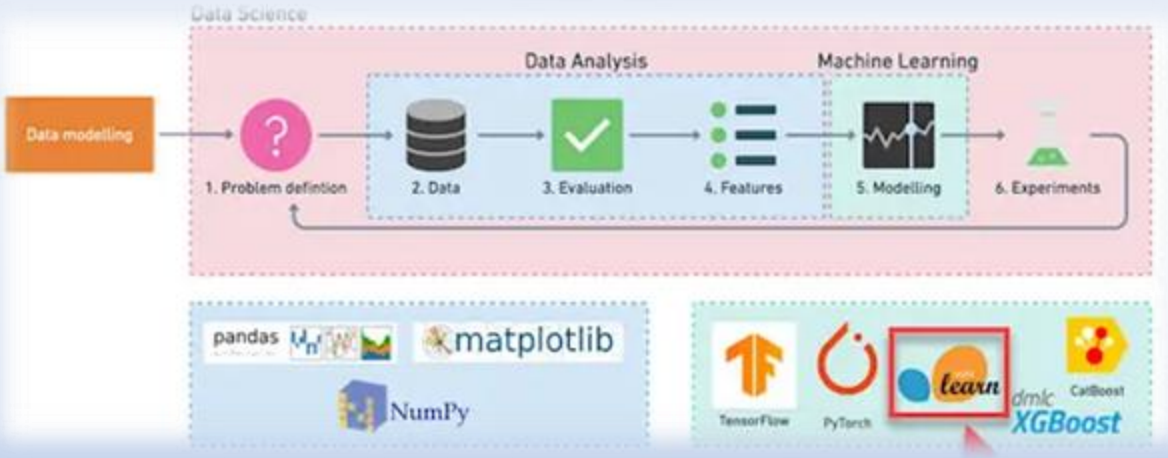
Chương 5.

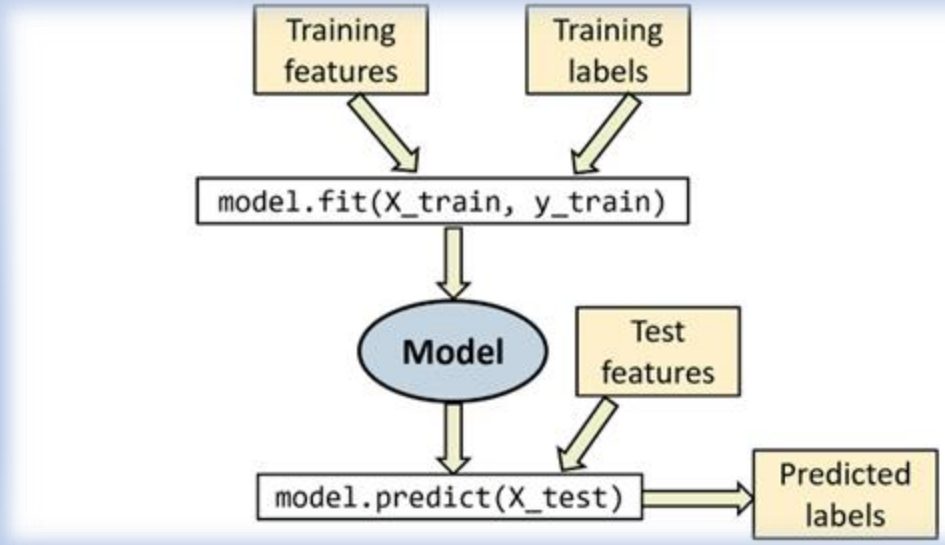
Modeling – Xây dựng mô hình

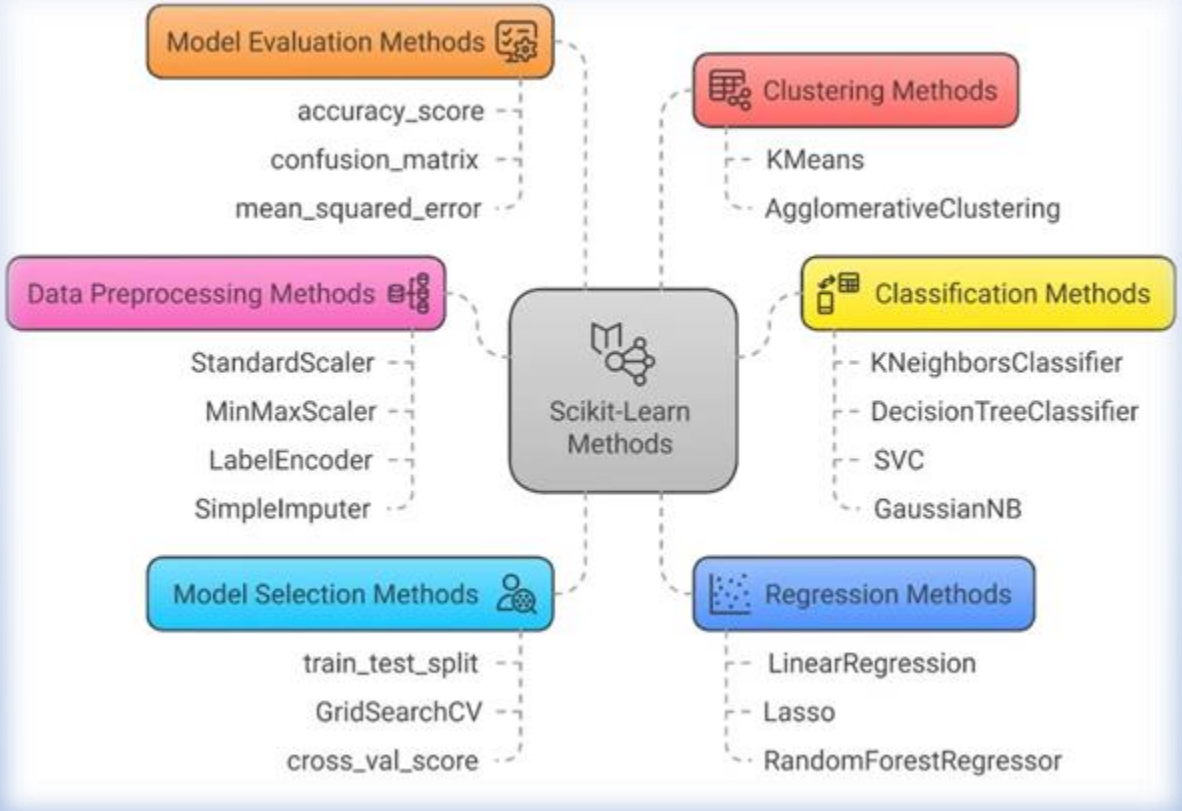


PhD. Nguyễn Thị Khánh Tiên
tiennkt@ut.edu.vn

Scikit-learn Methods for ML

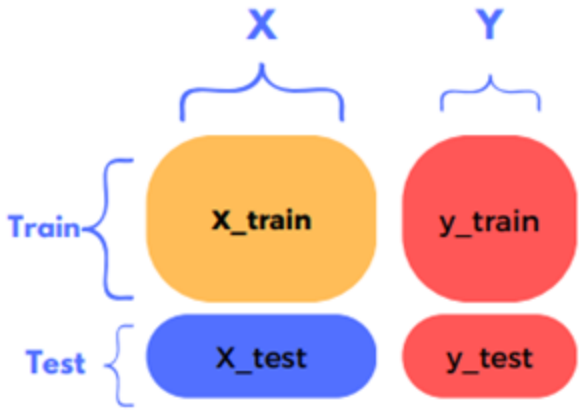






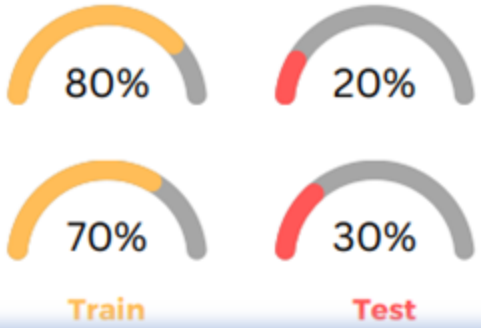
Scikit-learn Methods for Supervised Learning

Train-Test Split



- **Method of dividing the data into two part: Training and Testing**
- **Training Data : Model Training**
- **Testing Data : Model Evaluation**

Standard Ratio




```
from sklearn.model_selection import
train_test_split
X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3)
```

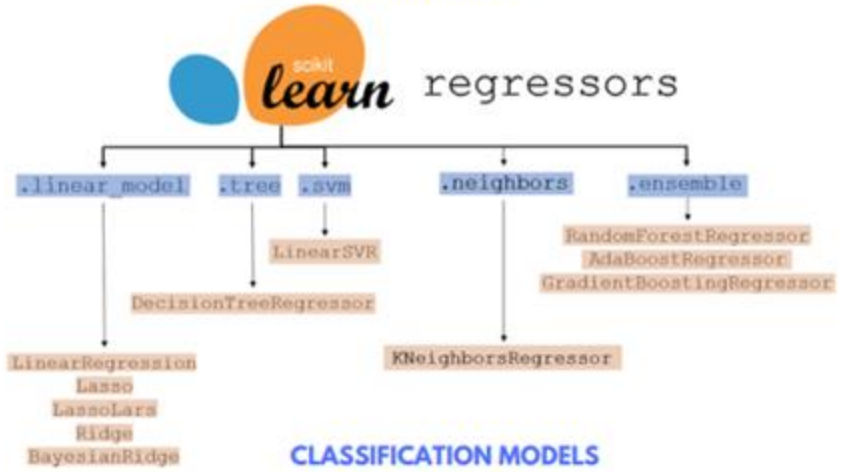
```
#Import
from sklearn.branch import
model_name

# Create a Instance
model = model_name()

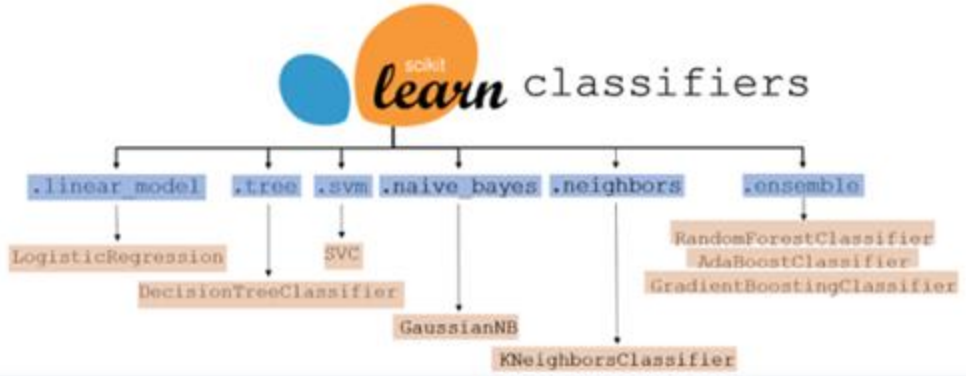
# fit model
model.fit(X_train, y_train)
```

Supervised Learning Models

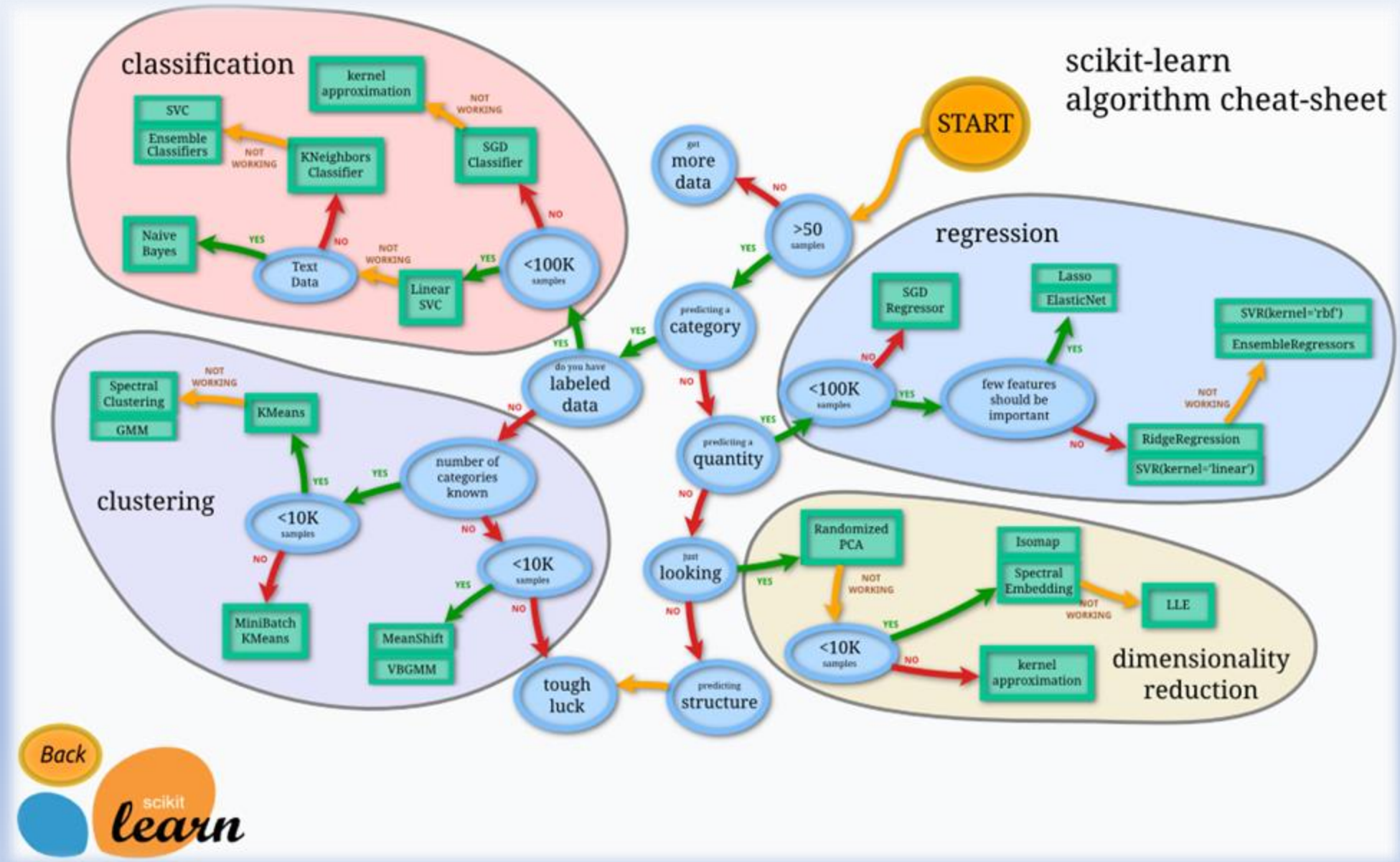
REGRESSION MODELS



CLASSIFICATION MODELS



Scikit-learn Algorithm



KDnuggets | Visit KDnuggets.com for more cheatsheets and additional learning resources.

Scikit-learn CheatSheet



Scikit-learn is an open-source Python library for all kinds of predictive data analysis. You can perform classification, regression, clustering, dimensionality reduction, model tuning, and data preprocessing tasks.

Loading the Data

Classification

```
from sklearn import datasets
X, y = datasets.load_wine(return_X_y=True)
```

Regression

```
diabetes = datasets.load_diabetes()
X, y = diabetes.data, diabetes.target
```

Training And Test Data

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)
```

Preprocessing the Data

Standardization

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_X_train = scaler.fit_transform(X_train)
scaled_X_test = scaler.transform(X_test)
```

Normalization

```
from sklearn.preprocessing import Normalizer
norm = Normalizer()
norm_X_train = norm.fit_transform(X_train)
norm_X_test = norm.transform(X_test)
```

Binarization

```
from sklearn.preprocessing import Binarizer
binary = Binarizer(threshold=0.0)
binary_X = binary.fit_transform(X)
```

Encoding Categorical Features

```
from sklearn.preprocessing import LabelEncoder
lab_enc = LabelEncoder()
y = lab_enc.fit_transform(y)
```

Imputer

```
from sklearn.impute import SimpleImputer
imp_mean = SimpleImputer(missing_values=0,
strategy='mean')
imp_mean.fit_transform(X_train)
```

Supervised Learning Model

Linear Regression

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

Support Vector Machines

```
from sklearn.svm import SVC
svm_svc = SVC(kernel='linear')
```

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
```

Unsupervised Learning Model

Principal Component Analysis

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
```

K Means

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=5, random_state=0)
```

Model Fitting

Supervised Learning

```
lr.fit(X_train, y_train)
svm_svc.fit(X_train, y_train)
```

Unsupervised Learning

```
model = pca.fit_transform(X_train)
kmeans.fit(X_train)
```

Prediction

Supervised Learning

```
y_pred = lr.predict_proba(X_test)
y_pred = svm_svc.predict(X_test)
```

Unsupervised Learning

```
y_pred = kmeans.predict(X_test)
```

Evaluation

Accuracy Score

```
lr.score(X_test, y_test)

from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Mean Squared Error

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, y_pred)
```

R2 Score

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

Adjusted Rand Index

```
from sklearn.metrics import adjusted_rand_score
adjusted_rand_score(y_test, y_pred)
```

Cross-Validation

```
from sklearn.model_selection import cross_val_score
cross_val_score( lr, X, y, cv=5, scoring='f1_macro')
```

Model Tuning

```
from sklearn.model_selection import GridSearchCV
parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
model = GridSearchCV(svm_svc, parameters)
model.fit(X_train, y_train)
print(model.best_score_)
print(model.best_estimator_)
```

Subscribe to KDnuggets News

Example: Iris Flower Classification with Scikit-learn

Problem: Predict the species of an Iris flower based on its sepal length, sepal width, petal length, and petal width. This is a multi-class classification problem.

Step 1: Load Data

Scikit-learn comes with several toy datasets, including the Iris dataset, which is perfect for demonstration.

- *load_iris()*: Fetches the Iris dataset.
- *iris.data*: Contains the feature measurements (sepal length, sepal width, petal length, petal width).
- *iris.feature_names*: Names of the features.
- *iris.target*: Contains the species labels (0, 1, 2 representing 'setosa', 'versicolor', 'virginica').
- *iris.target_names*: The actual names of the target classes.
- Convert *X* to a Pandas DataFrame and *y* to a *Pandas Series* for better readability and manipulation, although *NumPy* arrays work perfectly fine with Scikit-learn.

```python
# *** Step 1: Load Data ***
# Import necessary libraries
from sklearn.datasets import load_iris
import pandas as pd

# Load the Iris dataset
iris = load_iris()

# The data is stored in .data attribute
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# The target variable is stored in .target attribute
y = pd.Series(iris.target)

# Display the first 5 rows of the features
print("Features (X):")
print(X.head())

# Display the first 5 target values and their corresponding names
print("\nTarget (y) and Target Names:")
print(y.head())
print("Target names:", iris.target_names)
```

Example: Iris Flower Classification with Scikit-learn

Step 2: Split Data

- *train_test_split()*: This function randomly splits the data.
- *test_size=0.30*: 30% of the data will be allocated to the testing set, and the remaining 70% to the training set.
- *random_state=42*: This ensures that the split is the same every time you run the code, which is important for reproducibility.

```python
# *** Step 2: Split Data ***
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
# test_size=0.30 means 30% of the data will be used for testing
# random_state for reproducibility (same split every time)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.30,
                                                    random_state=42)

print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

Step 3: Preprocess Data

For this particular dataset and Logistic Regression, scaling might not have a dramatic effect because the features are already on a similar scale. However, it's a good practice, especially for algorithms sensitive to feature scales (like SVMs, KNNs, neural networks).

- *StandardScaler()*: Standardizes features by removing the mean and scaling to unit variance. This ensures all features contribute equally to the model.
- *fit_transform(X_train)*: Calculates the mean and standard deviation from *X_train* and then applies the transformation.
- *transform(X_test)*: Applies the *same* mean and standard deviation learned from *X_train* to *X_test*. You **never** fit a scaler on the test data.

```python
# *** Step 3: Preprocess Data ***
# Standardization is a common preprocessing step for many machine learning algorithms
# It scales the features to have a mean of 0 and a standard deviation of 1
from sklearn.preprocessing import StandardScaler

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform both training and testing data
# IMPORTANT: Fit only on training data to avoid data leakage from the test set
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFirst 5 rows of X_train (original):")
print(X_train.head())

print("\nFirst 5 rows of X_train_scaled (after standardization):")
print(X_train_scaled[:5])
```

Example: Iris Flower Classification with Scikit-learn

Step 4: Choose a Model

For a multi-class classification problem like Iris, Logistic Regression is a good starting point due to its simplicity and interpretability.

- *LogisticRegression()*: Instantiates the Logistic Regression classifier.
- *solver='liblinear'*: Specifies the algorithm to use for optimization. Different solvers are suitable for different dataset sizes and types.
- *random_state*: Ensures the model's internal random processes (if any) are reproducible.

Step 5: Train the Model

- *model.fit(X_train_scaled, y_train)*: This is where the magic happens! The model learns the patterns and relationships between the features (*X_train_scaled*) and the target labels (*y_train*).

Step 6: Make Predictions

- *model.predict(X_test_scaled)*: The trained model takes the scaled test features as input and outputs its predicted class for each sample

```python
# *** Step 4: Choose a Model ***
# For this example, we will use Logistic Regression as a classification model
# Logistic Regression is suitable for binary and multiclass classification problems
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# solver='liblinear' is a good choice for smaller datasets
# random_state for reproducibility of the model's internal randomness
model = LogisticRegression(solver='liblinear', random_state=42)

print("\nChosen Model:", model)
```

```python
# *** Step 5: Train the Model ***
# Fit the model on the training data
model.fit(X_train_scaled, y_train)

print("\nModel trained successfully!")
```

```python
# *** Step 6: Make Predictions ***
# Use the trained model to make predictions on the test set
y_pred = model.predict(X_test_scaled)
# print("\nPredictions on the test set:")
# print(y_pred)
print("\nFirst 10 actual labels (y_test):", y_test.values[:10])
print("First 10 predicted labels (y_pred):", y_pred[:10])
```

0.0s

Example: Iris Flower Classification with Scikit-learn

Step 7: Evaluate the Model

Evaluate how well your model performed on the test set using various metrics. For classification, common metrics include accuracy, precision, recall, and F1-score, often summarized in a classification report and a confusion matrix.

- *accuracy_score()*: Calculates the proportion of correctly classified samples.
- *classification_report()*: Provides a detailed summary of precision, recall, and F1-score for each class, as well as overall support.
- *Precision*: Of all samples predicted as a certain class, how many were actually that class?
- *Recall*: Of all samples that truly belong to a certain class, how many were correctly predicted as that class?
- *F1-Score*: The harmonic mean of precision and recall, providing a balanced measure.
- *confusion_matrix()*: A table that shows the number of correct and incorrect predictions for each class. It's excellent for understanding where your model makes mistakes.

```python
# *** Step 7: Evaluate the Model ***
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f}")

# Generate a classification report (precision, recall, f1-score per class)
class_report = classification_report(y_test, y_pred, target_names=iris.target_names)
print("\nClassification Report:\n", class_report)

# Generate a confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# Interpretation of Confusion Matrix:
# Rows represent true labels, columns represent predicted labels.
# conf_matrix[i, j] is the number of observations known to be in group i
# and predicted to be in group j.
# For example, in a 3x3 matrix:
# [[TN, FP, FP],
#  [FN, TP, FN],
#  [FN, FN, TN]]
# (This interpretation is more for binary. For multi-class, it's just true vs predicted counts)
print("\n--- Confusion Matrix Interpretation ---")
print("Rows: True Labels")
print("Columns: Predicted Labels")
print(pd.DataFrame(conf_matrix, index=iris.target_names, columns=iris.target_names))
```

Example: Iris Flower Classification with Scikit-learn

Step 7: Evaluate the Model

Evaluate how well your model performed on the test set using various metrics. For classification, common metrics include accuracy, precision, recall, and F1-score, often summarized in a classification report and a confusion matrix.

- *accuracy_score()*: Calculates the proportion of correctly classified samples.
- *classification_report()*: Provides a detailed summary of precision, recall, and F1-score for each class, as well as overall support.
- **Precision:** Of all samples predicted as a certain class, how many were actually that class?
- **Recall:** Of all samples that truly belong to a certain class, how many were correctly predicted as that class?
- **F1-Score:** The harmonic mean of precision and recall, providing a balanced measure.
- *confusion_matrix()*: A table that shows the number of correct and incorrect predictions for each class. It's excellent for understanding where your model makes mistakes.

```python
# *** Step 7: Evaluate the Model ***
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f}")

# Generate a classification report (precision, recall, f1-score per class)
class_report = classification_report(y_test, y_pred, target_names=iris.target_names)
print("\nClassification Report:\n", class_report)

# Generate a confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# Interpretation of Confusion Matrix:
# Rows represent true labels, columns represent predicted labels.
# conf_matrix[i, j] is the number of observations known to be in group i
# and predicted to be in group j.
# For example, in a 3x3 matrix:
# [[TN, FP, FP],
#  [FN, TP, FN],
#  [FN, FN, TN]]
# (This interpretation is more for binary. For multi-class, it's just true vs predicted counts)
print("\n--- Confusion Matrix Interpretation ---")
print("Rows: True Labels")
print("Columns: Predicted Labels")
print(pd.DataFrame(conf_matrix, index=iris.target_names, columns=iris.target_names))
```

Example: Iris Flower Classification with Scikit-learn

Step 8: Tune Hyperparameters

Hyperparameter tuning involves finding the best set of hyperparameters for your model to optimize its performance. *GridSearchCV* is a common method for this.

- *GridSearchCV*: Systematically searches through a predefined set of hyperparameter values, evaluates each combination using cross-validation, and selects the combination that yields the best performance.
- *param_grid*: A dictionary where keys are hyperparameter names and values are lists of values to try.
- *cv=5*: Performs 5-fold cross-validation, meaning the training data is split into 5 parts, and the model is trained 5 times, each time using a different part as a validation set.
- *scoring='accuracy'*: The metric used to evaluate the model during cross-validation.
- *grid_search.best_params_*: The hyperparameter combination that resulted in the best performance.
- *grid_search.best_score_*: The cross-validation score achieved with the best hyperparameters.
- *grid_search.best_estimator_*: The actual trained model with the best hyperparameters.

```python
# *** Step 8: Tune Hyperparameters ***
# Hyperparameter tuning can be done using techniques like Grid Search or Random Search.
from sklearn.model_selection import GridSearchCV

# Define the parameter grid to search
param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],  # Inverse of regularization strength
    'solver': ['liblinear', 'lbfgs']     # Algorithms to use in the optimization problem
}

# Initialize GridSearchCV
# estimator: The model to tune
# param_grid: The dictionary of hyperparameters to try
# cv: Number of cross-validation folds
# scoring: The metric to optimize (e.g., 'accuracy' for classification)
grid_search = GridSearchCV(LogisticRegression(random_state=42),
                           param_grid, cv=5, scoring='accuracy')

# Fit GridSearchCV to the scaled training data
grid_search.fit(X_train_scaled, y_train)

print("\nBest Hyperparameters found:", grid_search.best_params_)
print("Best Cross-validation Accuracy:", grid_search.best_score_)

# Get the best model
best_model = grid_search.best_estimator_

# Evaluate the best model on the test set
y_pred_tuned = best_model.predict(X_test_scaled)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
print(f"Accuracy with Tuned Model on Test Set: {accuracy_tuned:.4f}")

# You can also get a classification report for the tuned model
print("\nClassification Report with Tuned Model:\n",
      classification_report(y_test, y_pred_tuned, target_names=iris.target_names))
```

Phân loại (Classification)

Phân loại (Classification) là một kỹ thuật **học có giám sát (supervised learning)** được sử dụng để **dự đoán nhãn hoặc loại (category/class)** của các điểm dữ liệu mới dựa trên **tập dữ liệu huấn luyện đã được gán nhãn**.

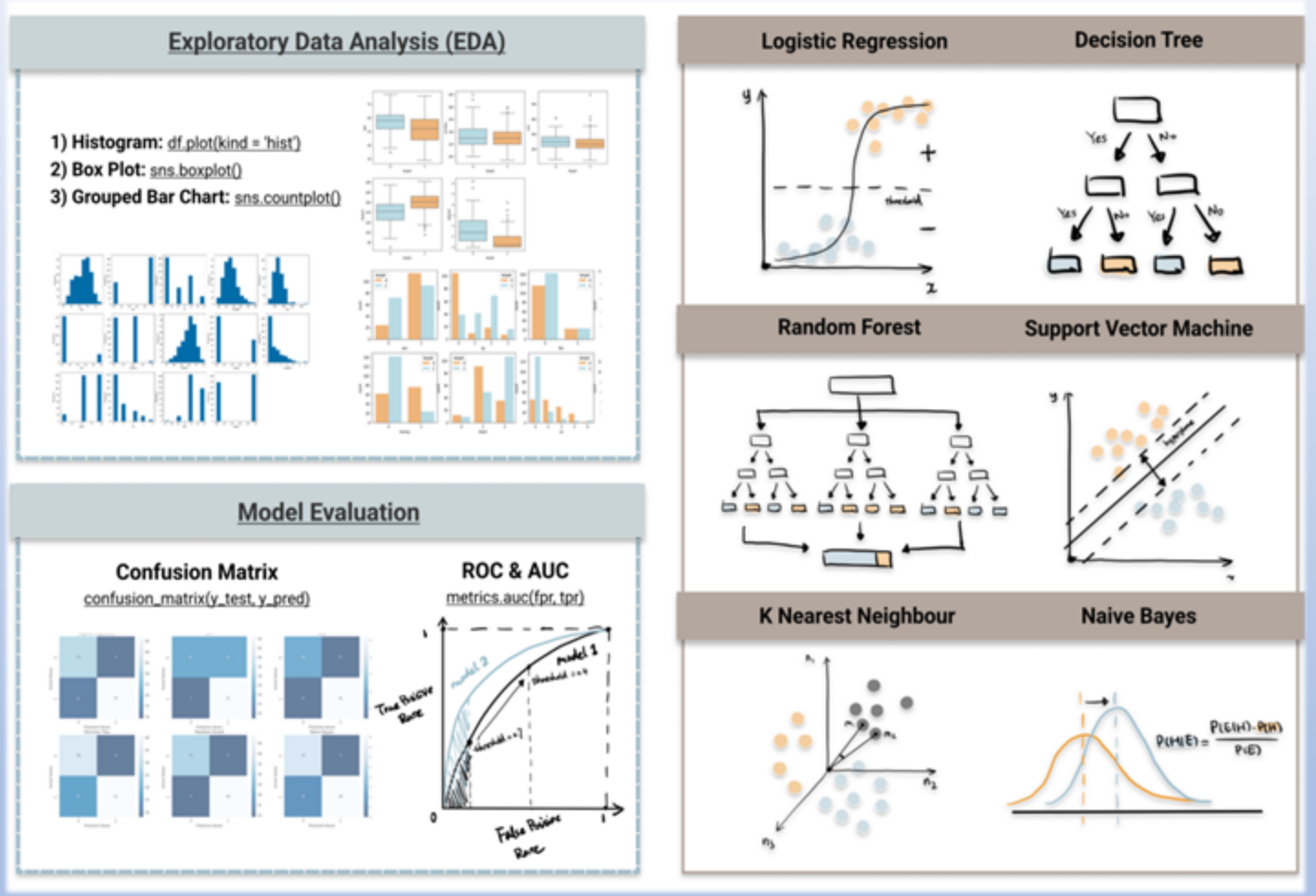
Trong quá trình phân loại, mô hình sẽ **học các mẫu (patterns)** từ dữ liệu huấn luyện để **phân loại các quan sát mới** vào **những nhóm hoặc lớp đã được xác định trước**.

Hai loại bài toán phân loại chính:

- **Phân loại nhị phân (Binary Classification):** Dự đoán kết quả chỉ có **hai lớp** khả thi (ví dụ: thư rác hoặc không, nam hoặc nữ, có hoặc không).
- **Phân loại đa lớp (Multi-class Classification):** Dự đoán kết quả có **hơn hai lớp** (ví dụ: phân loại các loại cây trồng, phân loại thể loại âm nhạc).

Ứng dụng của các thuật toán phân loại

- **Y tế (Healthcare):** Chẩn đoán bệnh dựa trên triệu chứng hoặc kết quả xét nghiệm.
- **Tài chính (Finance):** Phát hiện gian lận trong giao dịch.
- **Marketing:** Phân khúc khách hàng theo hành vi hoặc đặc điểm.
- **Xử lý ảnh (Image Processing):** Nhận dạng đối tượng trong hình ảnh.



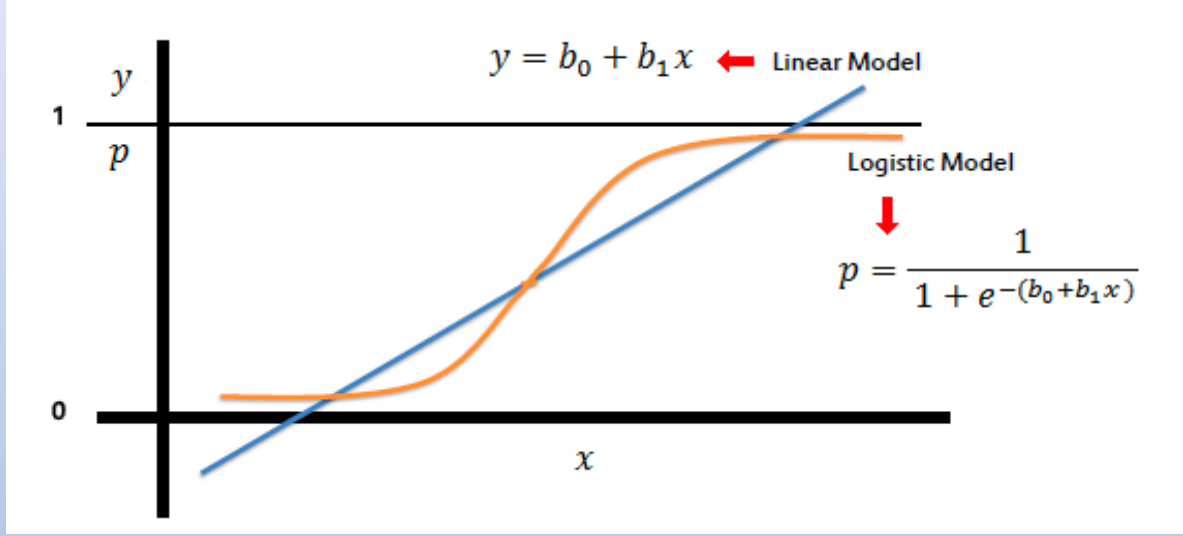
Các thuật toán Phân loại (Classification Algorithms)

(Decision Tree, Random Forest, Logistic Regression, SVM).

Thuật toán	Nguyên lý hoạt động	Ưu điểm	Nhược điểm	Ứng dụng tiêu biểu
Decision Tree (Cây quyết định)	Chia dữ liệu theo các điều kiện tại từng nút để tạo thành cây ra quyết định	- Dễ hiểu, trực quan - Không cần chuẩn hóa dữ liệu	- Dễ bị **overfitting** nếu cây quá sâu - Nhạy cảm với dữ liệu nhiễu	- Dự đoán rủi ro tín dụng - Phân loại khách hàng - Chẩn đoán bệnh
Random Forest (Rừng ngẫu nhiên)	Kết hợp nhiều cây quyết định huấn luyện trên các mẫu dữ liệu ngẫu nhiên, kết quả được lấy bằng **bỏ phiếu** hoặc **trung bình**	- Độ chính xác cao, ổn định - Giảm overfitting so với Decision Tree	- Khó diễn giải - Tốn tài nguyên khi huấn luyện	- Phát hiện gian lận - Phân loại văn bản - Dự báo tài chính
Logistic Regression (Hồi quy Logistic)	Dùng **hàm sigmoid** để mô hình hóa xác suất một quan sát thuộc về một lớp	- Đơn giản, dễ triển khai - Dễ diễn giải kết quả	- Chỉ phù hợp với dữ liệu **tuyến tính** - Hiệu suất thấp với dữ liệu phức tạp	- Dự đoán khách hàng mua hàng - Phân loại email spam - Dự đoán khả năng mắc bệnh
SVM (Support Vector Machine)	Tìm **siêu phẳng tối ưu** để phân tách các lớp dữ liệu, tối đa hóa khoảng cách giữa chúng	- Hiệu quả với dữ liệu có biên rõ ràng - Xử lý tốt dữ liệu **phi tuyến** với kernel	- Huấn luyện chậm với dữ liệu lớn - Khó tinh chỉnh siêu tham số	- Nhận dạng khuôn mặt - Phân loại văn bản - Nhận dạng chữ viết tay

Logistic Regression

Logistic regression is a statistical model used to predict the probability of a binary outcome (e.g., yes/no, true/false, 1/0). It's a widely used technique in various fields, including machine learning, statistics, and data science.



Applications of Logistic Regression:

- Predicting customer churn: Determining whether a customer is likely to stop using a product or service.
- Credit scoring: Assessing the risk of a loan default.
- Medical diagnosis: Predicting the presence or absence of a disease.
- Email spam filtering: Identifying spam emails based on their content.
- Sentiment analysis: Determining the sentiment (positive, negative, or neutral) of a text.

In scikit-learn,

- class ***sklearn.linear_model.LogisticRegression***

→https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression

- **Example:** Logistic Regression 3-class Classifier

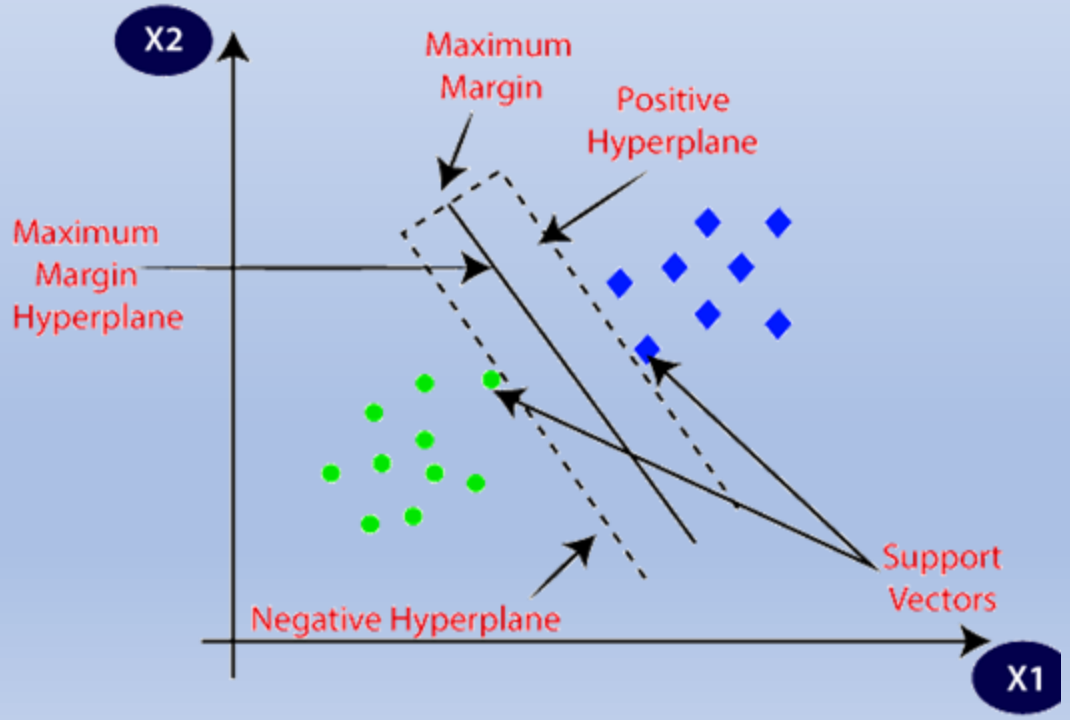
→https://scikit-learn.org/stable/auto_examples/linear_model/plot_iris_logistic.html#sphx-glr-auto-examples-linear-model-plot-iris-logistic-py

Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are a powerful machine learning algorithm used for classification and regression tasks. They are particularly effective in high-dimensional spaces and are known for their ability to handle complex decision boundaries.

Hyperplane: In SVM, the hyperplane is the optimal decision boundary that best separates the classes.

Support Vectors: The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector.



Kernel Functions: The choice of kernel function determines the type of mapping into the higher-dimensional feature space. Common kernel functions include:

- **Linear kernel**
- **Polynomial kernel**
- **Radial basis function (RBF) kernel**

In Scikit-learn, SVMs for classification - **SVC**, **NuSVC** and **LinearSVC**

→ https://scikit-learn.org/stable/modules/svm.html#

Example:

1. Plot different SVM classifiers in the iris dataset

→https://scikit-learn.org/stable/auto_examples/svm/plot_iris_svc.html#sphx-glr-auto-examples-svm-plot-iris-svc-py

2. SVM with custom kernel

→https://scikit-learn.org/stable/auto_examples/svm/plot_custom_kernel.html#sphx-glr-auto-examples-svm-plot-custom-kernel-py

3. RBF SVM parameters

→https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

Decision Tree Classification

Decision Trees are a popular machine learning algorithm often used for both classification and regression tasks.

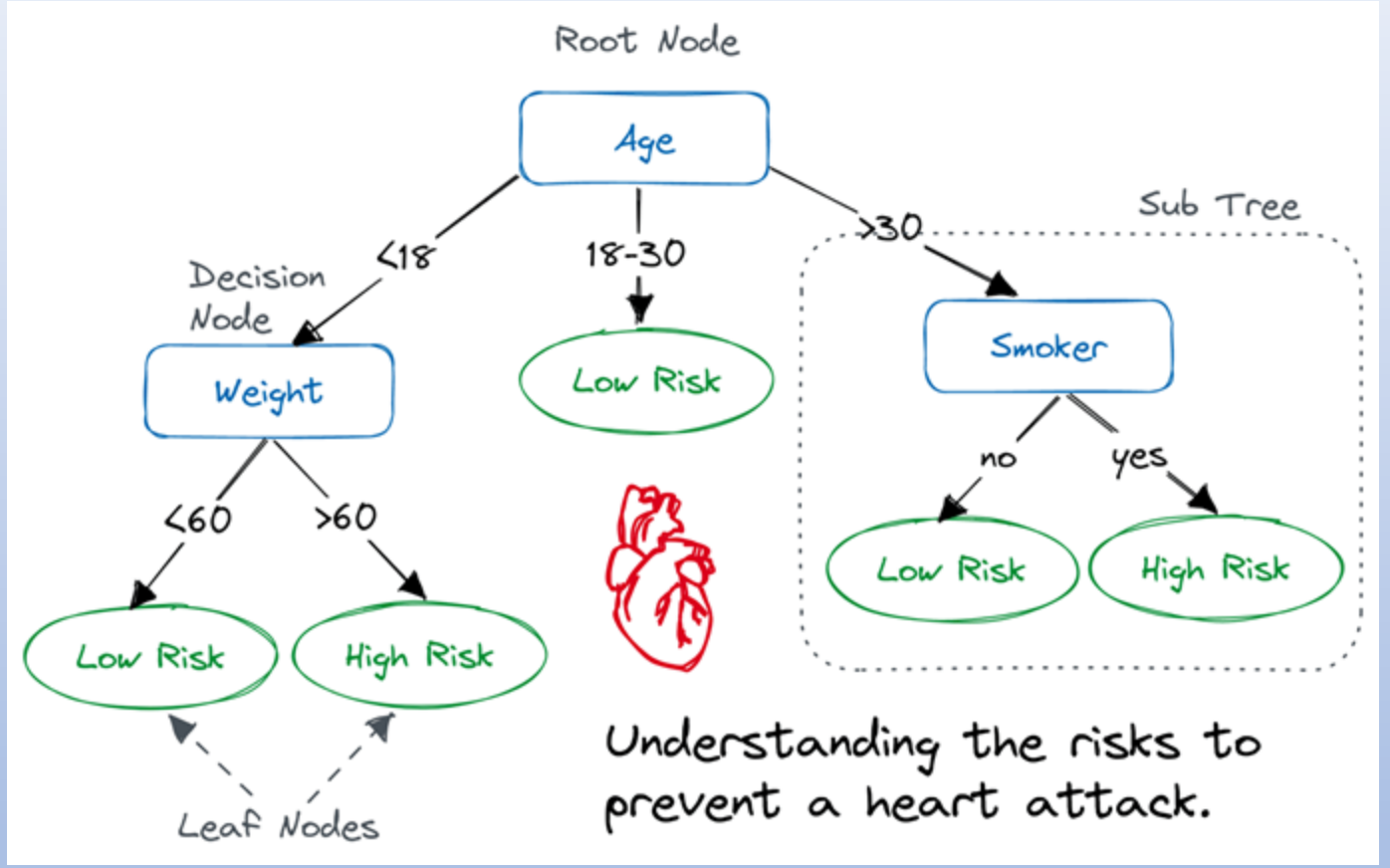
In the context of classification, they create a tree-like model where each internal node represents a test on an attribute (e.g., "Is age greater than 30?"), each branch represents the possible outcomes of the test, and each leaf node represents a class label.

The decision tree is a distribution-free or non-parametric method which does not depend upon probability distribution assumptions.

Decision trees can handle high-dimensional data with good accuracy.

Practice:

- → https://www.datacamp.com/tutorial/decision-tree-classification-python



Random Forest Classification

Random Forest is a powerful ensemble learning method that combines multiple decision trees to make predictions. It's particularly effective for classification tasks due to its ability to handle large datasets, reduce overfitting, and provide feature importance.

How Does Random Forest Work?

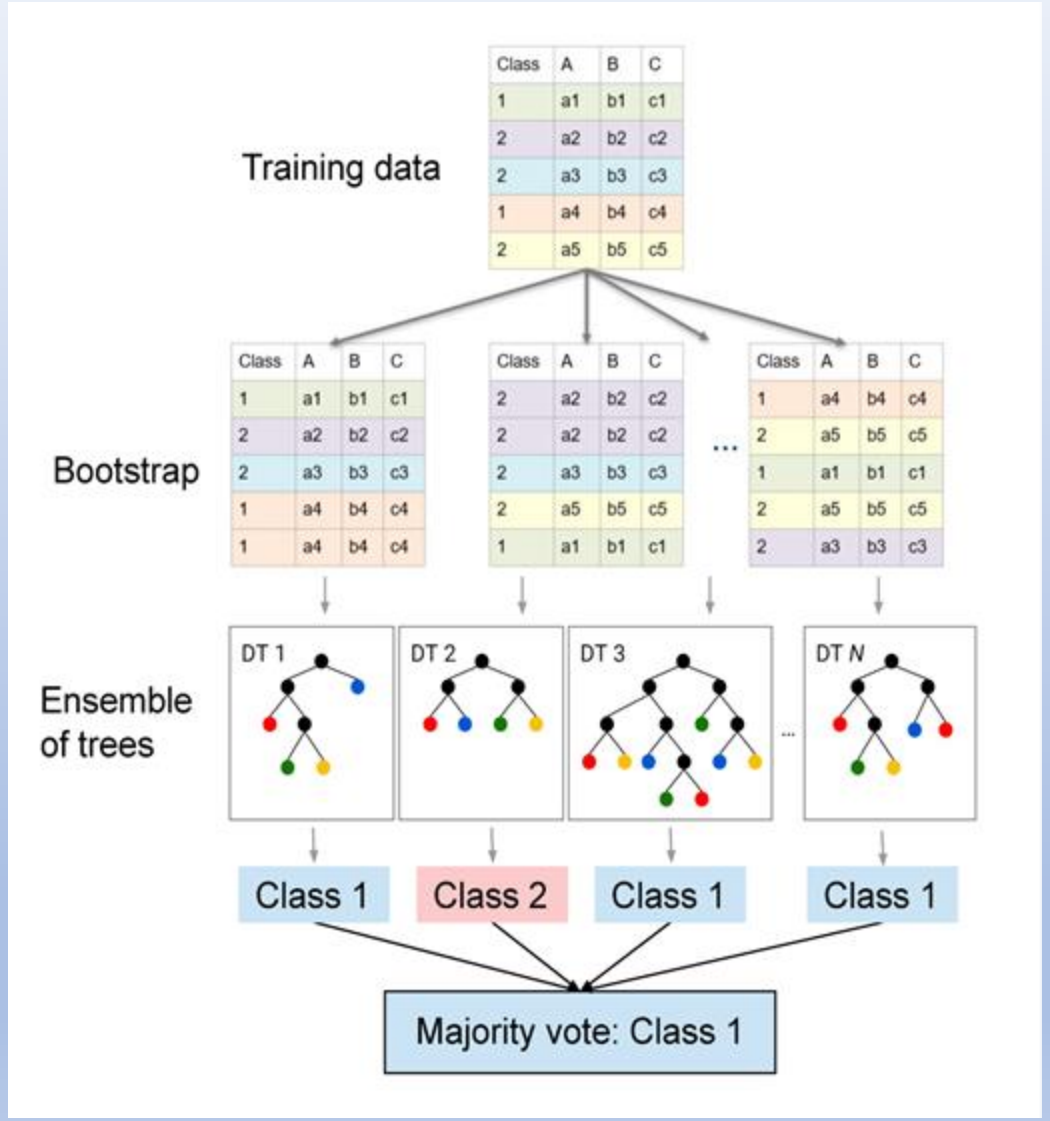
- **Bootstrap Sampling:** The algorithm randomly selects subsets of data from the original dataset with replacement. This creates multiple "bootstrap samples."
- **Decision Tree Growth:** Each bootstrap sample is used to grow a decision tree. The trees are grown to their maximum size without pruning.
- **Prediction:** To make a prediction for a new data point, the algorithm passes it through each decision tree and collects the predicted class from each.
- **Voting:** The most frequent class among all the predictions from the individual trees becomes the final prediction.

Key Parameters in Random Forest

- **Number of Trees:** The number of decision trees in the forest.
- **Maximum Depth:** The maximum depth of each decision tree.
- **Minimum Samples Split:** The minimum number of samples required to split an internal node.
- **Minimum Samples Leaf:** The minimum number of samples required to be at a leaf node.
- **Bootstrap:** Whether bootstrap sampling is used.

Practice:

→ https://www.datacamp.com/tutorial/random-forests-classifier-python



TÀI LIỆU THAM KHẢO

1. Han, Kamber & Pei – *Data Mining: Concepts and Techniques*.
2. Géron, Aurélien – *Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow*.
3. IBM – *CRISP-DM Process Model*.
4. scikit-learn.org – *Model Selection and Evaluation Guide*.
5. Kaggle datasets cho thực hành mô hình hóa.